UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE
SEDE SANTO DOMINGO DE LOS TSÁCHILAS

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN



PERIODO : 202650 abril 2026 – agosto 2026

ASIGNATURA : Modelos discretos para la ing

TEMA : Aplicaciones de árboles en sistemas computacionales

SUBTEMA : Complejidad algorítmica.

ESTUDIANTE : Jeffry Vera, Darlin Demera

NIVEL-PARALELO - NRC: 3vo ITIN

DOCENTE : Javier José Cevallos Farías

FECHA DE ENTREGA : 13/07/2026

SANTO DOMINGO - ECUADOR

Introducción

Dentro de los modelos discretos que se estudian en la carrera de Ingeniería en Tecnologías de la Información, el árbol es probablemente el que más se repite en la práctica profesional sin que el estudiante lo note. Cada vez que se abre un explorador de archivos, se ejecuta una consulta SQL o se comprime un video para subirlo a internet, hay un árbol trabajando por detrás. Esa omnipresencia es justamente lo que motiva este trabajo: entender por qué una estructura tan simple un nodo raíz del que cuelgan otros nodos termina resolviendo problemas tan distintos entre sí.

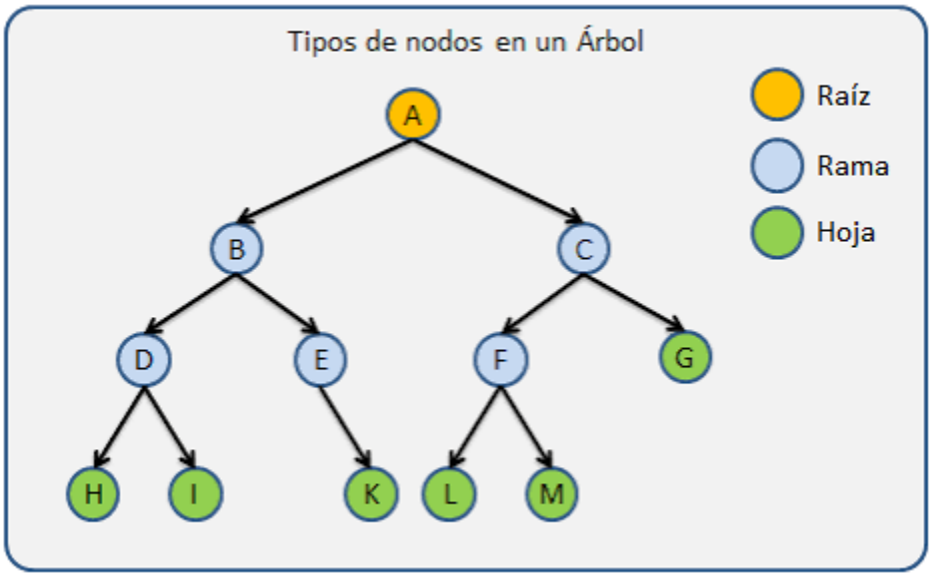
Un árbol organiza la información en niveles conectados por relaciones de padre e hijo, a diferencia de listas o vectores que solo permiten un recorrido lineal. Cuando la estructura se mantiene balanceada, buscar, insertar o eliminar un elemento toma un tiempo proporcional al logaritmo del número de elementos almacenados, en vez del tiempo lineal que tomaría revisar una lista completa. Esa sola ventaja explica gran parte de su éxito en sistemas donde el volumen de datos es enorme.

Este ensayo revisa cinco aplicaciones concretas de los árboles: la indexación en bases de datos mediante árboles B y B+, la clasificación automática mediante árboles de decisión, la representación de programas dentro de un compilador mediante árboles de sintaxis abstracta, la compresión de archivos mediante el árbol de Huffman y el enrutamiento de paquetes IP mediante estructuras tipo trie. Cada sección se apoya en fuentes académicas y técnicas especializadas y, al final, se plantea un análisis crítico sobre lo que estas aplicaciones dicen acerca del papel de los modelos discretos en la ingeniería de software.

Desarrollo

1. Fundamentos de los árboles como estructura de datos

Formalmente, un árbol es un caso particular de grafo: acíclico y conexo, con un nodo raíz del que se desprenden subárboles unidos mediante aristas hacia los nodos hijos. En la práctica de la programación se manejan varias familias: los árboles binarios de búsqueda (BST), los balanceados como AVL o Rojo-Negro, y los multivía como el árbol B, que se explica en la siguiente sección. Lo que comparten casi todas las aplicaciones que se revisan a continuación es una misma idea de fondo: mientras el árbol se mantenga balanceado, buscar, insertar o eliminar un dato cuesta un tiempo logarítmico y no lineal, y esa diferencia es la que en la práctica separa un sistema que responde en milisegundos de uno que se congela con datasets grandes.

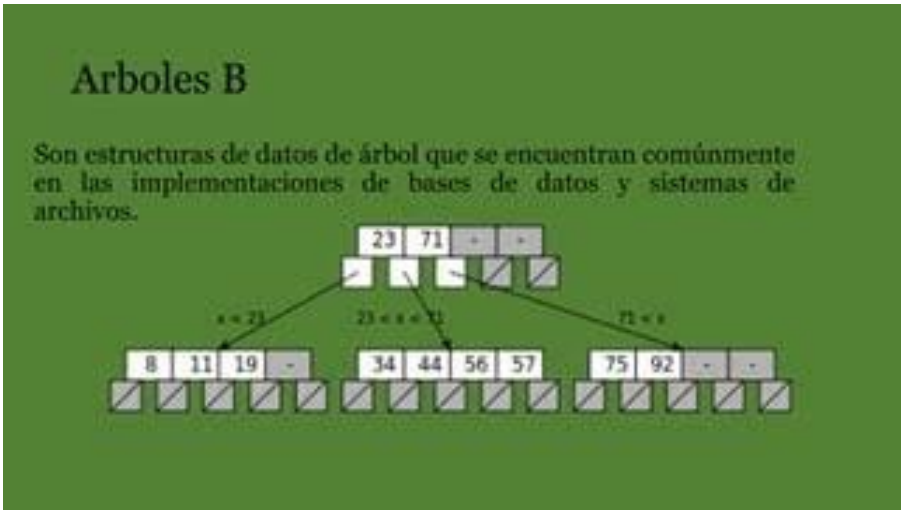


2. Árboles en la gestión de bases de datos: el árbol B y el árbol B+

El árbol B apareció en 1970, cuando Bayer y McCreight propusieron una forma de organizar índices grandes para que pudieran mantenerse ordenados sin perder eficiencia (como documenta Comer, 1979). Desde entonces se ha convertido en la estructura de indexación por defecto en la mayoría de los sistemas gestores de bases de datos relacionales. Comer (1979) lo describe, de hecho, como una estructura ubicua: presente por igual en dispositivos portátiles y en granjas de servidores, décadas antes de que existieran los teléfonos inteligentes.

A diferencia de un árbol binario, cada nodo de un árbol B puede tener decenas o cientos de hijos, lo que reduce mucho la altura del árbol y, con ella, el número de accesos a disco que exige localizar un registro. Motores tan usados como InnoDB de MySQL, PostgreSQL, Oracle y SQL Server emplean variantes B+ para sus índices primarios y secundarios; en InnoDB en particular, la tabla completa se guarda físicamente como un árbol B+ ordenado por la clave primaria, de modo que las búsquedas por esa clave son casi inmediatas (Nguyen, 2026). En un caso de estudio sobre PostgreSQL, Aminuddin et al. (2020) comprobaron que indexar con árboles B mejora de forma clara el rendimiento de las consultas frente a recorrer la tabla secuencialmente, sobre todo en búsquedas por rango.

La misma lógica se repite fuera de las bases de datos: sistemas de archivos como NTFS, ext4 y APFS usan estructuras parecidas a los árboles B+ para indexar directorios y metadatos, lo que confirma que un mismo principio discreto termina resolviendo problemas de organización en capas muy distintas de un sistema computacional.



3. Árboles de decisión en inteligencia artificial y aprendizaje automático

En inteligencia artificial, el árbol de decisión es uno de los modelos de clasificación más usados, en buena parte porque es fácil de interpretar: no es una caja negra. Organiza la clasificación como una secuencia de preguntas encadenadas, donde cada nodo interno prueba un atributo de entrada, cada rama corresponde a un resultado posible de esa prueba y cada hoja entrega finalmente una categoría o un valor de salida.

Entre los algoritmos que construyen estos árboles están ID3, C4.5, C5.0 y CART, además de métodos de conjunto como random forest y gradient boosting, que combinan varios árboles para ganar precisión. Sharma y Singh (2025) señalan que estos algoritmos han ganado terreno dentro del aprendizaje automático justamente porque logran un equilibrio entre simplicidad e interpretabilidad que modelos más opacos, como las redes neuronales profundas, no ofrecen; según los mismos autores, sus usos van desde la banca y el comercio minorista hasta la salud, pasando por la minería de texto y la mejora de motores de búsqueda.

Un caso aplicado a la educación es el de Kabakus y Kara (2021), quienes construyeron un modelo predictivo basado en árboles de decisión para estimar el desempeño académico de

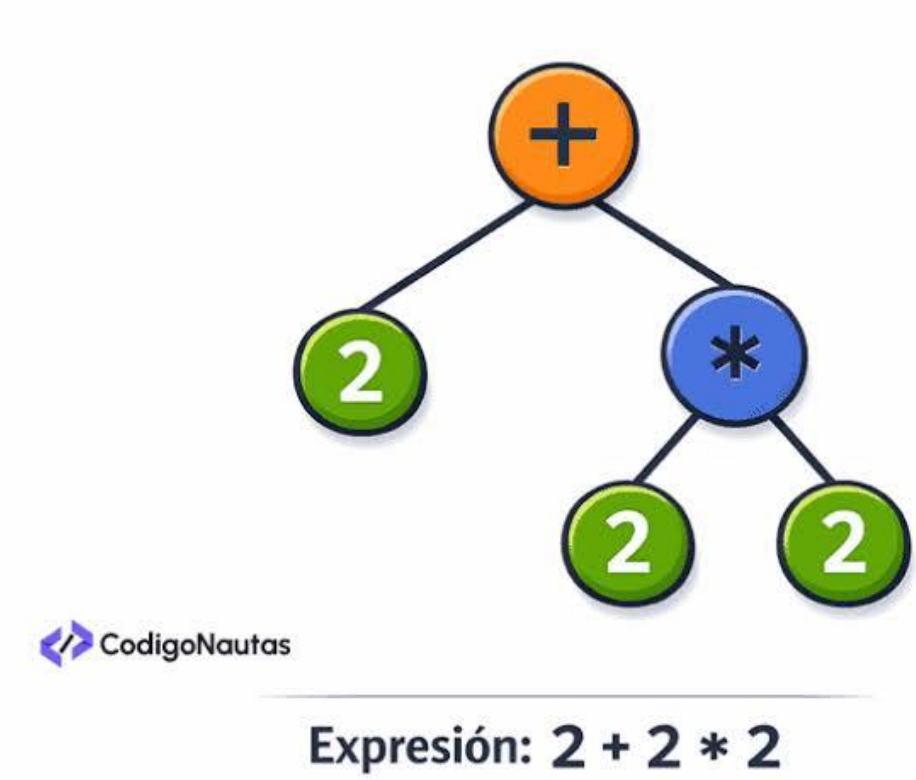
estudiantes en entornos de aprendizaje móvil, generando evaluaciones dinámicas ajustadas al perfil de cada uno. El ejemplo es útil porque muestra que un modelo tan sencillo en su lógica puede terminar sosteniendo un sistema de decisión con efecto directo sobre la vida académica de una persona.



4. Árboles de sintaxis abstracta en compiladores e intérpretes

En el diseño de compiladores e intérpretes, el árbol de sintaxis abstracta (AST, por sus siglas en inglés) es la estructura que representa la lógica de un programa una vez terminada la fase de análisis sintáctico. Conviene distinguirlo del árbol de análisis o parse tree, que conserva todos los detalles de la gramática paréntesis, comas, símbolos redundantes; el AST, en cambio, descarta ese ruido sintáctico y deja solo lo que importa semánticamente: los nodos internos son operadores y los nodos hoja son operandos (Dubach, 2021).

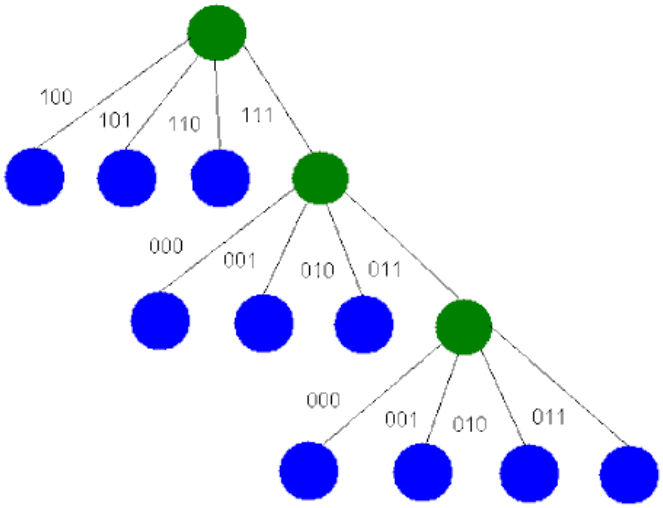
Esa simplificación es lo que permite que las fases posteriores del compilador análisis semántico, optimización, generación de código trabajen sobre una estructura navegable en lugar de sobre texto plano. No es un concepto que se quede en la pizarra: Hatakeyama (2025) explica cómo el compilador de TypeScript construye un AST antes de verificar tipos y transpilar a JavaScript, cómo Babel recorre y modifica su propio AST para generar código compatible con navegadores antiguos, y cómo linters como ESLint aplican reglas de análisis estático directamente sobre los nodos del árbol para detectar errores antes de que el programa se ejecute. En conjunto, estos tres casos muestran que el AST es la base operativa de herramientas que un desarrollador usa todos los días, no solo una construcción teórica de un curso de compiladores.



5. Árboles binarios en la compresión de datos: la codificación de Huffman

La compresión de datos sin pérdida es otro terreno clásico para los árboles, a partir del algoritmo que David A. Huffman publicó en 1952 sobre la construcción de códigos de redundancia mínima (Huffman, 1952). La idea es construir un árbol binario a partir de la frecuencia con la que aparece cada símbolo en un archivo: los símbolos menos frecuentes se combinan primero en nodos internos, y ese proceso voraz se repite hasta llegar a un único nodo raíz. Recorrer después el árbol de la raíz a cada hoja —anotando 0 al ir a la izquierda y 1 al ir a la derecha produce un código binario de longitud variable en el que los símbolos frecuentes quedan con códigos cortos y los raros con códigos largos.

Han pasado más de setenta años desde su publicación y la técnica sigue viva en formatos que se usan a diario, como ZIP, GZIP, JPEG y MP3. La investigación tampoco se detuvo: Żukowski (2026) propone una variante llamada PivCo-Huffman, construida sobre árboles wavelet, que gana velocidad de decodificación aprovechando operaciones vectorizadas (SIMD). Que un problema resuelto en los años cincuenta todavía reciba propuestas nuevas dice algo sobre lo bien planteado que estaba el problema original.



6. Árboles tipo trie en redes de computadoras y sistemas de búsqueda

En redes de computadoras aparece una variante especializada del árbol, el trie o árbol de prefijos, que se usa para resolver el enrutamiento de paquetes IP mediante la técnica llamada coincidencia de prefijo más largo (longest prefix match). En un trie de enrutamiento cada nodo representa un bit de la dirección IP, y el camino desde la raíz hasta un nodo dado representa un prefijo de red; al recorrer el árbol siguiendo los bits de la dirección de destino de un paquete, el enrutador encuentra el prefijo más específico que coincide con esa dirección.

Esta es la razón por la que el núcleo de Linux usa un trie, y no una tabla hash, para sus tablas de enrutamiento: un hash solo sirve para coincidencias exactas, mientras que el trie encuentra de forma natural la coincidencia más larga sin tener que probar cada longitud de prefijo por separado (Totoro Jam, 2026). En el terreno de la investigación, Wang et al. (2020) proponen con FlashTrie una estructura de trie comprimida con funciones hash capaz de sostener búsquedas por encima de los 100 Gbps en tablas con cientos de miles de prefijos IPv6, lo que muestra que optimizar árboles de prefijos sigue siendo un problema abierto en redes de alta velocidad. Fuera del enrutamiento, los tries también sostienen sistemas de autocompletado y corrección ortográfica, donde el objetivo es el mismo: encontrar rápido todas las palabras que comparten un prefijo dado.



Análisis crítico

Como grupo, lo que más nos llamó la atención al investigar estas cinco aplicaciones es que la fuerza de los árboles no está en su complejidad matemática —que en realidad es bastante modesta si se compara con otras estructuras combinatorias— sino en lo bien que un mismo principio se adapta a problemas que en apariencia no tienen nada en común. Un árbol B acelera el acceso a disco en una base de datos, un árbol de decisión estructura el razonamiento de un modelo predictivo, un AST representa la lógica de un programa, un árbol de Huffman reduce el tamaño de un archivo y un trie acelera el enrutamiento de millones de paquetes por segundo. Debajo de las cinco soluciones hay la misma jugada: partir un problema grande en subproblemas más pequeños organizados en niveles, para no tener que recorrer todo linealmente.

Esto tiene una consecuencia práctica para cualquiera que estudie ingeniería de software: aprender árboles en Modelos Discretos no es memorizar una estructura de datos aislada para aprobar el curso, es adquirir una herramienta que después reaparece en bases de datos, en ciberseguridad, en inteligencia artificial y en el desarrollo de compiladores, entre otras áreas. Se nota además que las optimizaciones más recientes PivCo-Huffman o FlashTrie, por ejemplo no cambian la idea original del árbol, sino que la ajustan a nuevas restricciones de hardware; el diseño de fondo sigue siendo el mismo que se planteó hace décadas, lo que dice bastante sobre qué tan bien resuelto estaba el problema desde el principio.

Dicho esto, los árboles tienen un límite claro: cuando las relaciones entre elementos no son jerárquicas sino que se cruzan entre sí como en una red social o en un sistema de recomendación, la herramienta correcta ya no es un árbol sino un grafo general. Reconocer esa frontera es, a nuestro parecer, tan importante como conocer las aplicaciones mismas: un

ingeniero que sabe elegir el modelo discreto adecuado para cada problema es más valioso que uno que solo sabe aplicar la misma estructura a todo.

Conclusiones

Los árboles son un modelo discreto que atraviesa casi toda la computación moderna, aunque no siempre se note a simple vista. En bases de datos y sistemas de archivos, los árboles B y B+ hacen posible indexar y recuperar información con rapidez incluso en volúmenes de datos enormes. En inteligencia artificial, los árboles de decisión ofrecen modelos predictivos interpretables que se usan tanto para clasificar como para explicar por qué se llegó a una decisión. En compiladores, el árbol de sintaxis abstracta permite representar y transformar programas de forma estructurada, y sostiene herramientas tan cotidianas como TypeScript, Babel o ESLint.

En compresión de datos, el árbol de Huffman sigue presente en formatos que se usan a diario, y en redes de computadoras los tries resuelven el enrutamiento de paquetes a velocidades de decenas de gigabits por segundo mediante coincidencia de prefijo más largo. Vistas en conjunto, estas cinco aplicaciones muestran que una idea matemática relativamente sencilla, bien combinada con un diseño algorítmico adecuado, puede sostener buena parte de la infraestructura computacional actual. De ahí que dominar los árboles como estructura de datos no sea un requisito más del pénsum, sino una base que un futuro ingeniero en tecnologías de la información va a seguir usando en cualquier área hacia la que se especialice.

Referencias

Aminuddin, A., Zaman, H. B., & Ahmad, K. (2020). A case study on B-tree database indexing technique. Journal of Soft Computing and Data Mining, 1(1), 27–35.

Comer, D. (1979). The ubiquitous B-tree. ACM Computing Surveys, 11(2), 121–137. https://doi.org/10.1145/356770.356776

Dubach, C. (2021). Compiler design lecture 7: Parse tree and abstract syntax [Diapositivas de clase]. McGill University, School of Computer Science. https://www.cs.mcgill.ca/~cs520/2021/slides/7-ast.pdf

Hatakeyama, A. (2025). The understanding abstract syntax trees (AST): How modern tools parse, analyze, and transform your code. Medium. https://medium.com/@dtianshan7/the-understanding-abstract-syntax-trees-ast-how-modern-tools-parse-analyze-and-transform-your-c3edc7e1e687

Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. Proceedings of the IRE, 40(9), 1098–1101. https://doi.org/10.1109/JRPROC.1952.273898

Kabakus, A. T., & Kara, R. (2021). Decision tree learning through a predictive model for student academic performance in intelligent m-learning environments. Computers and Education: Artificial Intelligence, 2, 100022. https://doi.org/10.1016/j.caeai.2021.100022

Nguyen, M. (2026). B-tree and B+ tree: Core data structures for database indexing. Medium. https://medium.com/@minhnguyenn/b-tree-and-b-tree-core-data-structures-for-database-indexing-cdc3c1868fb3

Sharma, R., & Singh, A. (2025). A survey of decision trees: Concepts, algorithms, and applications. IEEE Access, 12. https://ieeexplore.ieee.org/document/10562290/

Totoro Jam. (2026). Pattern: Trie (prefix tree). Battle-Tested Patterns. https://totoro-jam.github.io/battle-tested-patterns/patterns/trie/

Wang, C., Hu, C., & Liu, B. (2020). FlashTrie: Hash-based prefix-compressed trie for IP route lookup beyond 100 Gbps. Journal of Network and Computer Applications. https://www.researchgate.net/publication/224137076

Żukowski, M. (2026). PivCo-Huffman (versión 1.0) [Preprint]. arXiv. https://arxiv.org/pdf/2606.05765